

Succinct Representation of Data Structures

37.1	Introduction	37-1
37.2	Bitvector	37-2
37.3	Succinct Dictionaries	37-4
	Indexable Dictionary • Fully Indexable Dictionary • Dynamic Dictionary	
37.4	Tree Representations	37-7
	Binary Trees • Ordinal Trees • Cardinal Trees • Dynamic Binary Trees	
37.5	Graph Representations	37-12
37.6	Succinct Structures for Indexing	37-12
37.7	Permutations and Functions	37-14
	Permutations • Functions	
37.8	Partial Sums	37-16
37.9	Arrays	37-17
	Resizable Arrays • Dynamic Arrays	
37.10	Conclusions	37-19

J. Ian Munro
University of Waterloo

S. Srinivasa Rao
University of Waterloo

37.1 Introduction

Although computer memories, at all levels of the hierarchy, have grown dramatically over the past few years, increased problem sizes continues to outstrip this growth. Minimizing space is crucial not only in keeping data in the fastest memory possible, but also in moving it from one level to another, be it from main memory to cache or from a web site around the world. Standard data compression, say Huffman code or grammar based code, applied to a large text file reduces space dramatically, but basic operations on the text require that it be fully decoded.

In this chapter we focus on representations that are not only terse but also permit the basic operations one would expect on the underlying data type to be performed quickly. Jacobson [33] seems to have been the first to apply the term *succinct* to such structures; the goal is to use the information-theoretic minimum number of bits and to support the expected operations on the data type in optimal time. Our archetypical example (discussed in Section 37.4) is the representation of a binary tree. Suppose, we would like to support the operations of navigating through a binary tree moving to either child or the parent of the current node, asking the size of the subtree rooted at the current node or giving the unique ‘number’ of the node so that data can be stored in that position of an array. At $\lg n$ bits per reference, this adds up to at least $5n \lg n$ bits. However, there are only

$\binom{2n+1}{n}/(2n+1)$ binary trees, so the information-theoretic minimum space is fewer than $2n$ bits. Our archetypical data structure is a $2n + o(n)$ -bit representation that supports the operations noted above, and others, in constant time.

We consider a variety of abstract data types, or combinatorial objects, with the goal of producing such succinct data structures. Most, though not all, of the structures we consider are static. In most cases the construction of a succinct data structure from the standard representation is fairly straightforward in linear time.

Memory Model: We study the problems under the RAM model with word size $\Theta(\lg n)$, where n is the input size of the problem under consideration. This supports arithmetic (addition, subtraction, multiplication and division), indexing and bit-wise boolean operations (AND, OR, NOT, XOR etc.) on words, and reading/writing of words from/to the memory in constant time.

37.2 Bitvector

A bitvector provides a simple way to represent a set from any universe that is easily mapped onto $[m]^*$. Membership queries (checking whether a given element from the universe is present in the set) can be answered in constant time (in fact a single bit probe) using a bitvector. Furthermore, one can easily support updates (inserting and deleting elements) in constant time. The most interesting twist on the bitvector came with Jacobson [33] considering two more operations:

- **rank(i)** : return the number of 1s before the position i , and
- **select(i)** : return the position of the i -th 1.

As we shall see, these operations are crucial to a number of more complex structures supporting a variety of data types. An immediate use is to support the queries:

- **predecessor(x)** : find the largest element $y \leq x$ in the set S ,
- **successor(x)** : find the smallest element $y \geq x$ in the set S .

Given a bitvector of length m , Jacobson [33] gave a structure that takes $o(m)$ bits of additional space and supports **rank** and **select** operations by making $O(\lg m)$ bit probes to the structure. On a RAM with word size $\Theta(\lg m)$ bits, the structure given by Munro [40] enhanced this structure and the algorithms to support the operations in $O(1)$ time, without increasing the space bound. We briefly describe the details of this structure.

The structure for computing **rank**, the **rank directory**, consists of the following:

- Conceptually break the bitvector into blocks of length $\lceil \lg^2 m \rceil$. Keep a table containing the number of 1s up to the last position in each block. This takes $O(m/\lg m)$ bits of space.
- Conceptually break each block into sub-blocks of length $\lceil \frac{1}{2} \lg m \rceil$. Keep a table containing the number of 1s within the block up to the last position in each sub-block. This takes $O(m \lg m / \lg m)$ bits.
- Keep a precomputed table giving the number of 1s up to every possible position in every possible distinct sub-block. Since there $O(\sqrt{m})$ distinct possible blocks, and $O(\lg m)$ positions in each, this takes $O(\sqrt{m} \lg m \lg \lg m)$ bits.

*for positive integers m , $[m]$ denotes the set $\{0, 1, \dots, m-1\}$

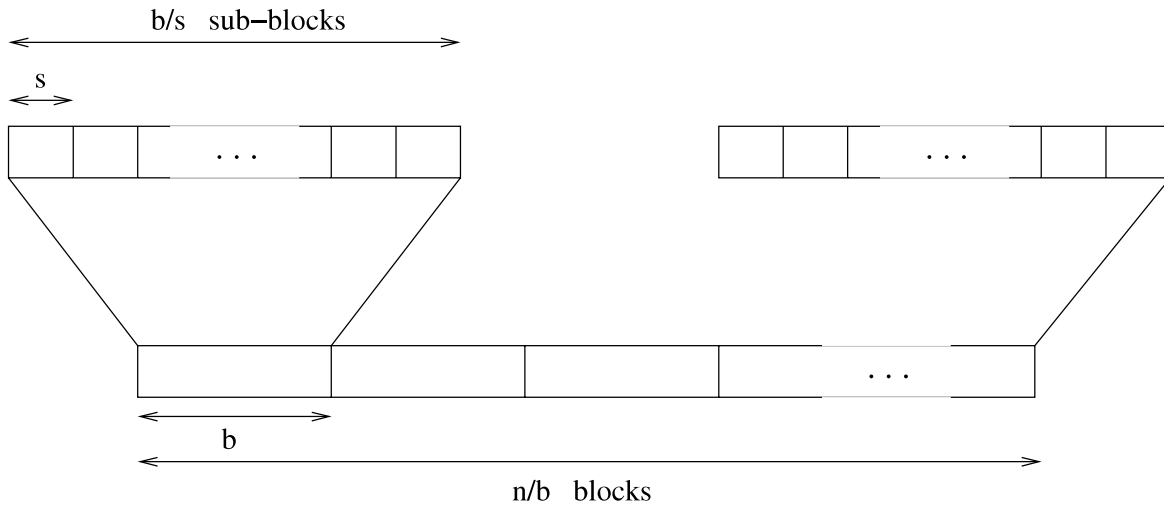


FIGURE 37.1: Two-level rank directory.

Thus, the total space occupied by this auxiliary structure is $o(m)$ bits. The **rank** of an element is, then, simply the sum of three values, one from each table.

The structure for computing **select** uses three levels of directories and is more complex. The first one records the position of every $(\lg m \lg \lg m)$ -th 1 bit in the bitvector. This takes $O(m/\lg \lg m)$ bits. Let r be the subrange between two values in the first directory, and consider the sub-directory for this range. If $r \geq (\lg m \lg \lg m)^2$ then explicitly store the positions of all ones, which requires $O(r/\lg \lg m)$ bits. Otherwise, subdivide the range and store the position (relative to the beginning of this range) of every $(\lg r \lg \lg m)$ -th one bit in the second level directory. This takes $O(r/\lg \lg m)$ bits for each range of size r , and hence $O(m/\lg \lg m)$ bits over the entire bitvector. After one more level of similar range subdivision, the range size will reduce to at most $(\lg \lg m)^4$. Computing **select** on these small ranges is performed using a precomputed table. The total space occupied by this auxiliary structure is $o(m)$ bits. The query algorithm is straightforward. See [9, 44] for details.

This ‘indexable bitvector’ is used as a substructure in several succinct data structures. To represent a bitvector of length m , it takes $m + o(m)$ bits of space. In general, if nothing is known about the bitvector then any representation needs at least m bits to distinguish between all possible bitvectors, and hence this is close to the optimal space. But if we also know the density (the number of ones) of the bitvector, then the space bound is no longer optimal, in general. The ‘fully indexable dictionary’ described in Section 37.3.2 gives a solution that takes nearly optimal space.

Using the ideas involved in constructing **rank** and **select** directories, one can also support the following generalizations of these two operations, using $o(m)$ bits of extra space: Given a bitvector of length m , and a fixed binary pattern p of length up to $(1 - \epsilon) \lg m$, for some fixed constant $0 < \epsilon < 1$

- $\text{rank}_p(i)$: return the number of (possibly overlapping) occurrences of p before the position i , and
- $\text{select}_p(i)$: return the i -th occurrence of the pattern p .

One can extend the ideas of **rank** and **select** directories to support indexing into a fixed or variable length encoded text (e.g. Huffman coding, prefix-free encoding etc.) in constant

time, using negligible extra space. See [33, 44] for some examples.

37.3 Succinct Dictionaries

The (static) dictionary problem is to store a subset S of size n so that membership queries can be answered efficiently. In our case, the universe is taken to be the set $[m]$. This problem has been widely studied and various solutions have been proposed to support membership queries in constant time.

As we have seen in the last section, a bitvector is a simple way of representing a set from a given universe. But this requires m bits of space. Since there are $\binom{m}{n}$ sets of size n from a universe of size m , one would require only $\mathcal{B} \equiv \lg \binom{m}{n}$ ($\approx n(\lg m - \lg n + \lg e)$, when $n = o(m)$) bits to store a canonical representation of any such set. Thus a bitvector is quite wasteful of space when the set is sparse. A sorted array is another simple representation, but it requires $\Theta(\lg n)$ time to answer queries. A *fusion tree* (see Chapter 39) also takes linear space and supports membership queries in $\Theta(\lg n / \lg \lg n)$ time. In this section, we consider representations of sets whose space complexity is close to the information theoretic minimum and support queries in constant time. (As all the structures outlined below support membership queries in worst case constant time, we do not mention the query complexity explicitly.)

Fredman, Komlós and Szemerédi [19] gave the first linear space structure for the static dictionary problem. This takes $n \lg m + O(n\sqrt{\lg n} + \lg \lg m)$ bits of space. The lower order term was later improved by Schmidt and Siegel [55] to $O(n + \lg \lg m)$. This structure uses a universe reduction function followed by a two-level hash function to hash the given subset one-to-one onto the set $[n]$, and stores the elements of subset in a hash table (in the order determined by the hash function). The hash table takes $n \lg m$ bits and a clever encoding of the hash function takes $O(n + \lg \lg m)$ bits of space. We refer to this as the FKS hashing scheme. Note that the space required for this structure is $\Theta(n \lg n)$ bits more than the optimal bound of $\mathcal{B}$ bits.

Brodnik and Munro [5] gave a static dictionary representation that takes $\mathcal{B} + o(\mathcal{B})$ bits of space. It uses two different solutions depending on the relative values of n and m . When the set is relatively sparse (namely, when $n \leq m/(\lg m)^{\lg \lg m}$), it partitions the elements into buckets based on the first $\lg n - \lg \lg m$ bits of their bit representations, and store explicit pointers to refer to the representations of individual buckets. Each bucket is represented by storing all the elements that fall into it in a perfect hash table for that bucket. Otherwise, when the set is dense, it uses two levels of bucketing (at each level splitting the universe into a number of equal-range buckets, depending only on the universe size) after which the range of these buckets reduces to $\Theta(\lg n)$. These small buckets are stored (almost) optimally by storing pointers into a precomputed table that contains all possible small buckets. In either case the space occupancy can be shown to be $\mathcal{B} + o(\mathcal{B})$ bits.

Pagh [46] observed that each bucket of the hash table may be resolved with respect to the part of the universe hashing to that bucket. Thus, one can save space by compressing the hash table, storing only the *quotient* information, rather than the element itself. From the FKS hash function, one can obtain a quotienting function that takes $\lg(m/n) + O(1)$ bits for each element. Using this idea one can obtain a dictionary structure that takes $n \lg(m/n) + O(n + \lg \lg m)$ bits of space, which is only $\Theta(n)$ bits more than the information-theoretic lower bound (except for the $O(\lg \lg m)$ term). Pagh has also given a dictionary structure that takes only $\mathcal{B} + o(n) + O(\lg \lg m)$ bits of space.

37.3.1 Indexable Dictionary

One useful feature of the sorted array representation of a set is that, given an index i , the i -th smallest element in the set can be retrieved in constant time. Furthermore, when we locate an element in the array, we immediately know its **rank** (the number of elements in the set which are less than the given element). On the other hand, hashing based schemes support membership in constant time, but typically do not maintain the ordering information. In this section we look at a structure that combines the good features of both these approaches.

An *indexable dictionary* is a structure representing a set S of size n from the universe $[m]$ to support the following queries in constant time:

rank(x, S): Given $x \in [m]$, return -1 if $x \notin S$, and $|\{y \in S \mid y < x\}|$ otherwise, and

select(i, S): Given $i \in \{1, \dots, n\}$, return the i -th smallest element in S .

Here the **rank** operation is only supported for the elements present in the set S . Ajtai [1] showed that the more general problem of supporting **rank** for every element in the universe has a query lower bound of $\Omega(\lg \lg n)$, even if the space used is polynomial in n . As a consequence, we emphasize the need for handling both S and its complement in the next section.

A dictionary that supports **rank** operation [52], as well as an indexable dictionary is very useful in representing trees [50] (see Section 37.4.3).

Elias [15] considered the indexable dictionary problem and gave a representation that takes $n \lg m - n \lg n + O(n)$ bits and supports the queries in $O(1)$ time, though he only considered the average case time complexity of the queries. Raman et al. [50] have given an indexable dictionary structure that takes $\mathcal{B} + o(n) + O(\lg \lg m)$ bits. The main idea here is, again, to partition the elements into buckets based on their most significant bits, as in the static dictionary structure of Brodnik and Munro [5]. The difference is that instead of storing explicit pointers to the bucket representations, they store the bucket sizes using a succinct representation that supports partial sum queries (see Section 37.8) in constant time. This not only saves a significant amount of space, but also provides the extra functionality needed for supporting **rank** and **select**.

Using similar ideas, one can also represent multisets and collections of sets using almost optimal space. See [50] for details.

37.3.2 Fully Indexable Dictionary

Given a set $S \subseteq [m]$, a *fully indexable dictionary* (FID) of S is a representation that supports **rank** and **select** operations on both S and its complement $\bar{S} = [m] \setminus S$ in constant time [50].

It is easy to see that the bitvector representation of a set, with auxiliary structures to support **rank** and **select** on both the bits as mentioned in Section 37.2, is an FID. But this requires $m + o(m)$ bits, where m is the size of the universe. Here we look at an FID representation that takes $\mathcal{B} + o(m)$ bits of space. Note that when the set is reasonably sparse (namely when $n = m/\omega(\lg m)$) $\mathcal{B} = o(m)$, and hence it improves the space complexity of the bitvector representation.

Let $S \subseteq [m]$ be a given set of size n . Divide $[m]$ into blocks of consecutive elements, with block size $u = \lfloor \frac{1}{2} \lg m \rfloor$. Let S_i be the subset of S that falls into the i -th block. Each of the S_i 's is represented by storing an index into a table that contains the characteristic bitvectors of all possible subsets of a particular size from the universe $[u]$. As a consequence, the space occupied by these representations together with all the precomputed tables can be shown to be $\mathcal{B} + o(m)$ bits. To enable fast access to the representations of these subsets, we store the partial sums of the sizes of the subsets, and also the partial sums of the lengths

of the representations of these subsets, which take $O(m \lg \lg m / \lg m)$ bits. This can be used to support **rank** in $O(1)$ time.

To support **select**, we first store the positions of every $(\lg^2 m)$ -th element explicitly in an array, which takes $O(m / \lg m)$ bits. Call the part the universe that lies between two successive elements in this array a *segment*. If the size of a segment is more than $\lg^4 m$, then we explicitly store all the $\lg^2 m$ elements of S that belong to this segment in sorted order. This takes $\lg^3 m$ bits for every such ‘sparse’ segment, and hence at most $m / \lg m$ bits, over all the sparse segments. Dense segments are handled by constructing a complete tree with branching factor $\sqrt{\lg m}$, and so constant height, whose leaves are the blocks that constitute this segment, and storing some additional information to navigate this tree efficiently (see the *searchable partial sum* structure in Section 37.8).

To support **rank** and **select** on $\bar{S}$, first observe that an implicit representation of a set over a given universe is also an implicit representation of its complement. Thus, we need not store the implicit representations of $\bar{S}_i$ again. Except for this, we repeat the above construction with S_i ’s replaced by $\bar{S}_i$ ’s.

The overall space requirement of the structure is $\mathcal{B} + O(m \lg \lg m / \lg m)$ bits, and **rank** and **select** are supported on both S and $\bar{S}$ in $O(1)$ time. See [50] for details.

37.3.3 Dynamic Dictionary

We have looked at several succinct structures for static dictionaries. We now briefly consider the dynamic dictionary problem where one can add and delete elements from the set while supporting the membership queries.

Model: The model of memory allocation is very important in dynamic data structures. One widely used model [4, 45, 50] is to assume the existence of a ‘system’ memory manager that would allocate and free memory in variable-sized chunks. In this model, the space complexity of a structure is counted as the total size of all the blocks allocated for that structure, and hence this approach does not account for the space wastage due to *external fragmentation*.

Fundamentally, memory is most easily viewed as a large array. If we are to use the storage, we must manage it. Therefore a simple view is to count all the fragmentation we may cause and count the memory usage as the difference between the addresses of the first and last locations used by the structure. While more complex scenarios may be more realistic in certain cases, we take this simple address difference model as our focus. The methods we discuss are equivalent under either model up to constant factors.

A balanced tree can be used to support all the dynamic dictionary operations in $O(\lg n)$ time using $n \lg m + O(n \lg n)$ bits, where n is the current size of the set. Using the ideas of the FKS dictionary, Dietzfelbinger et al. [14] gave a dynamic dictionary structure that supports membership in $O(1)$ time and updates (insert/delete) in $O(1)$ expected amortized time. This structure takes $O(n \lg m)$ bits of space. There have been several improvements, lowering the space complexity close to the information theoretic-minimum, culminating in a structure that takes $\mathcal{B} + o(\mathcal{B})$ bits with the same query complexity as above. See [5, 18, 46, 47, 51] and the references therein.

All these structures also support associating satellite information with the elements, so that whenever an element is found to be in the set, we can also retrieve the satellite information associated with it in constant time.

37.4 Tree Representations

Trees are one of the most fundamental objects in computer science. We consider the problem of representing large trees succinctly. Storing a tree with a pointer per child as well as other structural information can account for the dominant storage cost. For example, standard representations of a binary tree on n nodes, using pointers, take $O(n \lg n)$ bits of space. Since there are only $\binom{2n+1}{n}/(2n+1)$ different binary trees on n nodes, less than $2n$ bits suffice to distinguish between them. We look at some binary tree representations that take $2n + o(n)$ bits and support the basic navigational operations in constant time.

37.4.1 Binary Trees

First, if the tree is a complete binary tree (i.e., a binary tree in which every level, except possibly the deepest, is completely filled, and the last level is filled from the left as far as required), then there is a unique tree of a given size and we require no additional space to store the tree structure. In fact, by numbering the nodes from 1 to n in the ‘heap order’ [59] (left-to-right level-order traversal of the tree), one can support navigational operations on the tree by observing that the parent of a node numbered i is the node numbered $\lfloor i/2 \rfloor$, and the left and right children of node i are $2i$ and $2i + 1$ respectively. But this property does not hold when the tree is not complete.

If the tree is not complete, one could extend it to a complete binary tree with the same height and store a bit vector indicating which nodes are present in the tree (in the heap order of the complete tree) to support the operations efficiently. But this takes space exponential in the number of nodes, in the worst case.

To save space, one can use the following compressed representation due to Jacobson [33]: First, mark all the nodes of the tree with 1 bits. Then add external nodes to the tree, and mark them with 0 bits. Construct a bitvector by reading off the bits that are marking the nodes in left-to-right level-order. (See Figure 37.2.) It is easy to see that the original tree can be reconstructed from this bitvector. For a binary tree with n nodes, this bitvector representation takes $2n + 1$ bits. Moving between parent and child is just a slight twist on the method used in a heap. By storing the **rank** and **select** directories for this bitvector, one can support the navigational operations in constant time using the following equations:

$$\text{parent}(i) = \text{select}(\lfloor i/2 \rfloor); \quad \text{leftchild}(i) = 2 \cdot \text{rank}(i); \quad \text{rightchild}(i) = 2 \cdot \text{rank}(i) + 1.$$

37.4.2 Ordinal Trees

Now, consider optimal representations of trees of higher degree, of which there are two different notions.

An *ordinal tree* is a rooted tree of arbitrary degree in which the children of each node are ordered. Ordinal trees on n nodes are in one-to-one correspondence with binary trees on n nodes. Hence about $2n$ bits are necessary to represent an arbitrary ordinal tree on n nodes. A *cardinal tree* of degree k is a rooted tree in which each node has k positions for an edge to a child. Hence, a binary tree is a cardinal tree of degree 2. There are $C_n^k \equiv \binom{kn+1}{n}/(kn+1)$ cardinal trees of degree k on n nodes [25]. Hence we need roughly $(\lg(k-1) + k \lg \frac{k}{k-1})n$ bits to represent an arbitrary such tree.

The basic operations we would like to support on tree representations are: given a node, finding its parent, i -th child, degree and the size of the subtree rooted at that node (subtree size). For the cardinal trees we also need to support the additional operation of finding a child with a given label.

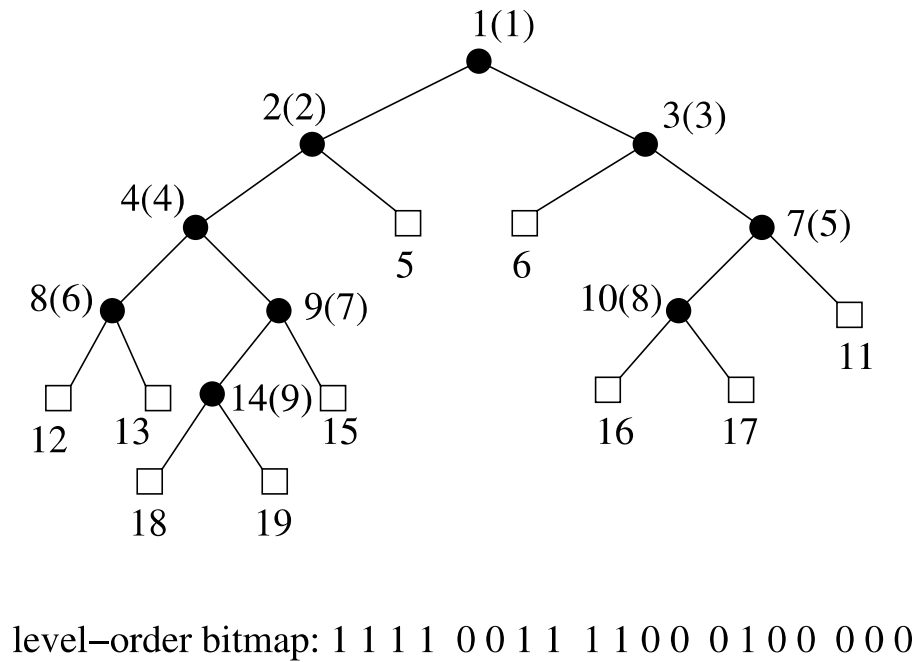


FIGURE 37.2: Level-order bitmap representation of a binary tree.

We outline three different representations of an ordinal tree. All the three representations map the n nodes of the tree onto the integers $1, \dots, n$, and hence all are appropriate for applications in which data is to associated with nodes or leaves.

Level-order unary degree sequence representation: A rooted ordered tree can be represented by storing its degree sequence in any of a number of standard orderings of the nodes. The ordinal tree encoding of Jacobson [33] represents a node of degree d as a string of d 1s followed by a 0. Thus the degree of a node is represented by a binary prefix code. These prefix codes are then written in a level-order traversal of the entire tree. Using auxiliary structures to support **rank** and **select** operations on this sequence, one can support finding the parent, the i -th child and the degree of any node in constant time. Thus, it gives a representation that takes $2n + o(n)$ bits of space and supports the above three operations in constant time, for an ordered tree on n nodes.

Balanced parenthesis representation: The tree encoding of Munro and Raman [41] uses a balanced sequence of parentheses to represent an ordinal tree. This balanced representation is derived from the depth-first traversal of the tree, writing an open parenthesis on the way down and a close parenthesis on the way up. Thus, a tree on n nodes can be represented by a balanced parenthesis sequence of length $2n$. Extending the ideas of Jacobson, they showed how to support the following operations in $O(1)$ time, using negligible extra space ($o(n)$ bits):

- **findopen/findclose(i):** find the position of the open/close parenthesis matching the given close/open parenthesis in position i .
- **excess(i):** find the difference between the number of open and closing parentheses before the position i .
- **enclose(i):** given a parenthesis pair whose open parenthesis is in position i , find the open parenthesis corresponding to its closest enclosing matching parenthesis pair.

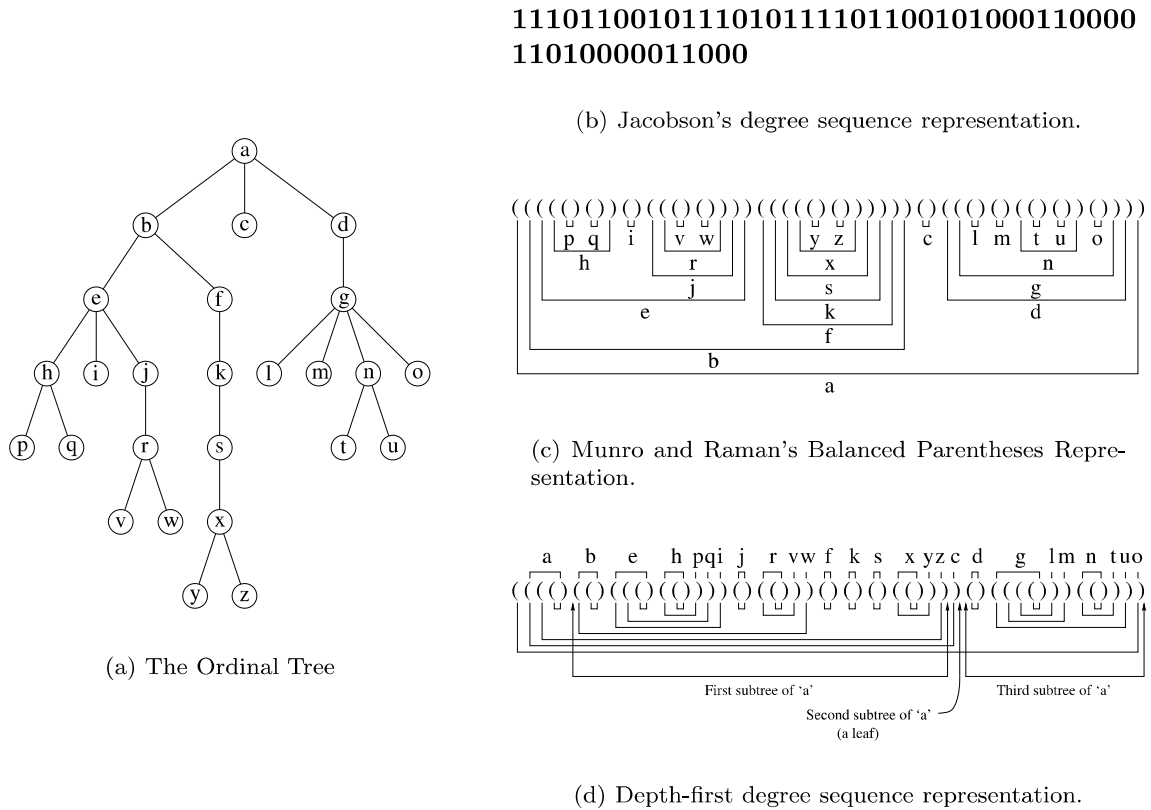


FIGURE 37.3: Three encodings of an ordinal tree.

The parent of a node can be found in constant time using the **enclose** operation. In the parenthesis representation, the nodes of a subtree are stored together, which enables us to support the operation of finding the size of the subtree rooted at a given node in constant time. The problem with this representation is that finding the i -th child takes $\Theta(i)$ time.

Depth-first unary degree sequence representation: Jacobson's representation allows access to the i -th child in constant time, whereas Munro and Raman's representation supports subtree size operation in constant time. To combine the virtues of these two representations, Benoit et al. [2] used a representation that writes the unary degree sequence of each node in the depth-first traversal order of the tree. The representation of each node contains essentially the same information as in Jacobson's level-order degree sequence, but written in a different order. Thus, it gives another $2n$ bit encoding of a tree on n nodes. Replacing the 0's and 1's by open and close parentheses respectively, and adding an extra open parenthesis at the beginning, creates a string of balanced parentheses. Using auxiliary structures to support **rank** and **select** operations on this bit string and also the operations on balanced parenthesis sequences defined above, one can support finding the parent, i -th child, degree and subtree size of a given node in constant time.

Other operations: Sadakane [54] has shown that the parenthesis representation of an ordinal tree can be used to support least common ancestor queries in $O(1)$ time using a $o(n)$ -bit auxiliary structure. Munro and Rao [42] have shown that one can also support the level ancestor queries in $O(1)$ time, using an additional $o(n)$ bit auxiliary structure by storing the parenthesis representation. Geary et al. [23] obtained another structure that takes $2n + o(n)$ bits and supports level-ancestor queries, in addition to all the other